Adaptive Seeds and Finite Quotients

An obstruction to smaller iPRA recursors,
with the current complete-checker benchmark

Research prepared for Vladimir Reshetnikov

November 10, 2026

Abstract

The requested complete axiomatization strictly below the current draft frontier has not been obtained. We give a broader negative result instead: computing a recursor's initializer by a fixed finite adaptive circuit of queries to its own update preserves a one-coordinate unary bound on every numerical term. This remains true with arbitrary fixed unary primitive recursive primitives and many bounded multivariate primitives.

We then close a substantial interpretation loophole. On any domain $D \subseteq \mathbb{N}^d$, even modulo an arbitrary equivalence relation, a binary operation represented coordinatewise by such terms cannot have a two-sided unit on an infinite quotient. A word of at most $\lceil \log_2(d+1) \rceil$ unit maps gives a uniformly bounded representative of every class. This word-length bound is sharp. The conclusion excludes finite-dimensional term interpretations with quotients, but not general relational interpretations.

The comparison is pinned to ProveIt commit `0300a290bedee5019c974291850b798378438b2e`. Across 16 current archives, including their embedded history, the attained coordinate minima are 672 normalized Python bytes, 277 significant tokens, and 311 AST nodes. Maximum-compression gzip sizes are also measured, strictly as information. An exact certificate demonstrates why deleting recursive major proofs loses a valid closure theorem. The report supplies complete proofs of the new obstruction, a fully explicit inherited native calculus, reproducible source accounting, and the precise remaining research obligations.

Status. No new complete checker below the current frontier, no absolute minimality theorem, and no Lean/Rocq formalization are claimed. The positive deliverable is a proved obstruction and an auditable research package. The requested stronger positive construction remains open here.

Contents

1 The result and its attribution 3

2 The actual current benchmark 3

3 The complete inherited native calculus 4

4 A general obstruction to computing the initializer from the update 6

5 Finite tuple domains and quotient equality 10

6 An exact obstruction to removing recursive major proofs 13

7 Validation, reproducibility, and limits 14

8 Further research questions 14

A Source map and exact benchmark programs 15

B Reproduction commands 16

1 The result and its attribution

The existing drafts have already reached a compact numerical basis and a small logical interface. Several recent implementations differ by only a few source characters or by a different treatment of malformed JSON. A valid improvement must preserve full intuitionistic first-order arithmetic, not only its numerical equations, and must establish both interpretation composites. Merely obtaining a shorter parser or an equiconsistent fragment would not meet the request.

This investigation makes three distinct contributions. First, it extends the particular initializer obstruction in the counter-congruence and transient-seed drafts to arbitrary fixed finite adaptive query circuits. Second, it proves a finite-quotient obstruction for every numerical term system with the resulting bounds; this covers finite tuple domains and nontrivial equality quotients. Third, it reproduces the actual current checker envelope and gives a concrete theorem lost by a tempting deletion of recursive elimination-premise checking.

The class of functions bounded by a unary function of one fixed input is an established notion: these are *almost unary* functions on a countable set, and their closure under composition is standard in clone theory [7]. The particular bound for $B(0;h) = 1 + h(0,0)$ and the no-addition/no-pairing corollaries already occur in the supplied drafts [2, 3]. We attribute those facts. The additional coverage proved here is the adaptive-initializer family and the finite-dimensional quotient argument, including its sharp logarithmic dependence bound. A literature-wide priority claim is not made.

2 The actual current benchmark

The reference is the repository snapshot

`0300a290bedee5019c974291850b798378438b2e`.

All 16 top-level ZIP archives in `docs/drafts` were retrieved. A recursive scan, deduplicating identical archive payloads, found 22 distinct archives, 516 Python source occurrences, and 297 distinct Python source hashes. A public-driver filter identifies 262 checker-candidate occurrences and 134 distinct candidate hashes. The filter is an inventory aid; minimum-attaining candidates were separately inspected. These numbers are not automatic certification of all candidate programs. The 23 distinct non-driver sources falling below at least one normalized coordinate were also inspected; none is a complete proof checker. Their individual classifications are included in the evidence files.

2.1 One cost boundary for every complete program

The inherited comparison convention removes comments and genuine docstrings, applies `ast.unparse`, appends one final newline, and counts normalized UTF-8 bytes, significant Python tokens, and AST nodes. The complete module is charged: JSON input, opening and scope handling, every inference branch, exception handling, and the public verdict driver. The Python interpreter and standard JSON library are shared infrastructure. The recorded runtime is CPython 3.12.14, with zlib 1.3.2.

Complete reference profile	Norm. bytes	Tokens	AST	Raw bytes	Gzip-9
Counter congruence: zero-product	**672**	281	322	672	397
Constructed computation: explicit	684	**277**	317	684	403
Transient seed: structure	705	280	**311**	705	393
Local rewriting: scalar	685	290	334	**628**	396
Local rewriting: balanced	736	286	320	667	**381**
Transient seed: joint	684	280	314	684	390

The last column compresses the *raw complete Python file*, including the gzip wrapper. Both raw and normalized compression measurements are included in the machine-readable inventory. The smallest normalized-source gzip size is 390 bytes, attained by the joint profile. Gzip is not an optimization objective in this work.

The exact compression expression is

```
gzip.compress(source_bytes, compresslevel=9, mtime=0)
```

The fixed timestamp removes one irrelevant source of variation. Compression implementation versions can still affect exact bytes.

The minima are attained by different programs. In the inherited three normalized coordinates, a single strict improvement must have fewer than 672 bytes, 277 tokens, and 311 AST nodes. No program produced here meets that condition. Mathematically, these attained sizes are *upper* bounds on an unknown optimal size. Calling the envelope a "lower bound" is understandable shorthand for the benchmark to beat, but not a proven impossibility bound for all checkers.

2.2 Qualifications that affect comparisons

The 672-byte leader occurs in an ancillary directory and implements an inherited theory. It must count even though it is not the main proposal in that archive. It uses the predicate `a*0==[]` to distinguish JSON-decoded lists in its declared runtime contract. This is a source spelling change, not a new arithmetic principle.

The 311-node and 381-byte gzip profiles use permissive exception-based openers. In particular, nonempty strings can reject through Python's recursion limit rather than through a structurally terminating malformed- input parser. These qualifications are disclosed in the source drafts. Such profiles must not silently be compared with a stronger arbitrary-JSON termination contract. The raw 628-byte minimum also must not be compared against another program's normalized byte count.

The full sources, original archive paths, hashes, and measurements are included. The reproduction program parses the source corpus; it does not execute arbitrary corpus files. Earlier snapshot figures in the retained reference report are historical and are superseded by this table.

3 The complete inherited native calculus

For clarity, this section specifies a complete small presentation already present in the corpus [4, 5]. It is the reference from which reductions were attempted, not a new axiomatization claimed by this report. The target is full first-order intuitionistic PRA. Atomic induction in the language containing

every fixed primitive-recursive function is a standard formulation [6]; ordinary quantifier-free induction is recovered using primitive-recursive characteristic functions.

3.1 Numerical syntax and nonlogical computation schemas

There is one numerical sort, with finite terms

$$t ::= x \mid 0 \mid St \mid I(n, a; v.h).$$

Only h binds v; its other variables are passive parameters. Every body is fixed earlier finite syntax. Equivalently, each body supplies a staged first-order function symbol, rather than a numerical argument to a uniform total evaluator. The only numerical computation schemas are

$$I(0, a; v.h) = a, \tag{I0}$$
$$I(Sn, a; v.h) = h[I(n, a; v.h)/v]. \tag{IS}$$

Later drafts also remove primitive S by a successor-seeded recursor; that reduction and its predecessor construction are inherited. It is not needed to state the present obstructions.

3.2 Formula syntax and every primitive inference

The public formula syntax is

$$A ::= (s = t) \mid Q_y A \mid D_{x,y} A.$$

A context Γ is a reusable finite list; the paired bound variables x, y are distinct. Substitutions are capture-free. The designated native absurdity is $0 = S0$. The ordinary readings are

$$(Q_y B)^\circ = B^\circ(0) \to \forall y\, B^\circ(y), \qquad (D_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y\, B^\circ(x, y)\big).$$

These explain the rules; a full converse interpretation still requires the logical and arithmetic bridge discussed below.

The following is the entire primitive rule inventory:

$$\frac{}{\Gamma \vdash A}\ (A \in \Gamma) \qquad \frac{\Gamma \vdash s = t \quad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \tag{Hyp, Tr}$$

$$\frac{\Gamma \vdash Q_y B(y) \quad \Gamma \vdash B(0)}{\Gamma \vdash B(t)} \tag{QE}$$

$$\frac{\Gamma \vdash D_{x,y} B(x, y) \quad \Gamma, Q_y B(a, y) \vdash A}{\Gamma \vdash A}. \tag{DE}$$

For DE, a is fresh for Γ, A, and all fixed parameters of the major. The template F in Tr is arbitrary; this does not authorize induction on an arbitrary formula.

The two paired-introduction forms are

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y} B(x, y)}, \qquad \frac{\Gamma, E(t, z) \vdash E(t, Sz)}{\Gamma \vdash D_{x,y} E(x, y)}. \tag{DI-case, DI-progress}$$

Here z is fresh for Γ, t and all fixed matrix parameters. The second matrix E must be a literal equation. The displayed local hypothesis is discharged. The first form permits every formula matrix, including equations.

Finally, a computation instance may be temporarily supplied:

$$\frac{\Gamma, E \vdash A}{\Gamma \vdash A} \quad (E \text{ is an instance of I0 or IS}). \tag{Comp}$$

There are no further primitive logical or arithmetic rules in this presentation. Computation as theorem leaves is equivalent: Comp is a cut with the theorem, while a computation theorem follows from Comp and Hyp. Weakening, contraction, exchange, renaming, and substitution are admissible operations on finite derivations, not an external uncharged proof engine.

3.3 What the bi-interpretation requires

In the direction to ordinary iPRA, the term I is an ordinary fixed primitive recursion. Tr is equality substitution, QE is implication and universal elimination, and DE is existential elimination. DI-case is proved by the numerical split $y = 0$ or $y = Sz$. DI-progress uses equation induction: its step proof and an assumed base establish every instance. Thus the restriction to an equation in DI-progress is essential.

The reverse direction is substantially more than these soundness checks. It must derive identity expansion, contextual numerical cases, equation induction, predecessor and selectors, formula gates, all intuitionistic connectives and both quantifiers, actual-equality reflection for quantifier-free predicates, and a compiler for every fixed primitive recursion. Finally it must prove the numerical and formula interpretation composites internally. The inherited construction keeps the whole numerical domain and actual equality; the numerical isomorphism is the identity. Matching standard numerical outputs alone would not suffice.

For comparison, the ordinary first-order realization uses intuitionistic equality logic, I0 and IS, literal-equation induction, numerical coverage $\forall x\,(x = 0 \vee \exists y\, x = Sy)$, and the explicit separation sentence $(0 = S0) \to \bot$. This ordinary inventory is separate from the native rules above. In particular, the genuine ordinary falsity symbol must not be silently identified with an arbitrary equation without the separation bridge.

The complete inherited development is retained, unchanged and attributed, as `reference/inherited_complete_presentation.tex`. The current report does not substitute finite tests for that metatheory, does not claim a new independent Lean/Rocq verification of it, and does not infer bi-interpretability merely from equiconsistency. Its new theorems below are standard-model obstructions with their own complete proofs.

4 A general obstruction to computing the initializer from the update

The current counter-congruence and transient-seed drafts already prove a one-coordinate majorization theorem for the particular operator

$$B(0; u, v.h) = 1 + h(0, 0), \qquad B(n+1; u, v.h) = h(n, B(n; u, v.h)).$$

The transient-seed draft also states composition closure and the nonexistence of an injective pairing term. We do not claim those conclusions again as new results. The following extension identifies a substantially broader family of initializer-elimination attempts to which the same obstruction applies.

It allows finitely many adaptive probes of the update and arbitrary primitive-recursive combinations of the answers. The restriction is that the initializer receives no independent unbounded numerical input.

Fixed query circuits. Fix a finite acyclic circuit P with no ordinary input nodes. Its nodes are numerals; applications of fixed primitive-recursive functions to earlier node values; or queries $H(a, b)$, whose two arguments are earlier node values. One node is designated as output. The circuit may query H several times, and a later query may depend on earlier answers. Its finite graph and all its ordinary primitive-recursive operations are fixed by the constructor, rather than supplied as numerical inputs at runtime.

For a fixed update term $h(u, v, \vec{p})$, let $h_{\vec{p}}(a, b) = h(a, b, \vec{p})$. Define the tied-seed recursor

$$\begin{aligned} R_P(0; u, v.h) &= P^{h_{\vec{p}}}, \\ R_P(n+1; u, v.h) &= h(n, R_P(n; u, v.h), \vec{p}). \end{aligned} \tag{Tied seed}$$

The only access of the circuit to $\vec{p}$ is through queries to $h_{\vec{p}}$. In particular it does not receive a separate initializer, the actual counter argument, or a passive numerical parameter as an ordinary input. Each fixed instance denotes a primitive-recursive function: the circuit is finite and the outer recursion is ordinary primitive recursion. No universal evaluator is introduced.

Permitted term primitives. The term language may contain constants, projections, and arbitrary fixed unary primitive-recursive functions. More generally, it may contain any fixed primitive-recursive operation f for which either a constant bound is known or there are a fixed input coordinate j and unary primitive-recursive function F satisfying

$$f(\vec{x}) \le F(x_j). \tag{One-coordinate bound}$$

Thus predecessor, zero tests, bounded Boolean tests, minimum, and one-sided data gates are all permitted. Ordinary finite substitution and arbitrary nesting of the displayed R_P constructors are allowed. The arbitrary combining operations inside a query circuit need not obey this bound; they receive only the circuit's internally generated values.

For example, a permitted initializer first computes $q = h(0, 0, \vec{p})$ and then returns

$$q + h(q, q+1, \vec{p})^2.$$

The second query is adaptive and the final combination can grow rapidly. The theorem still applies: the values cannot acquire two independent unbounded controlling coordinates from this one update.

Lemma 4.1 (A circuit cannot acquire an independent unbounded input)**.** *For any fixed query circuit P the following statements hold.*

1. *If $H(a, b) \le M$ for all a, b, then $P^H \le C_P(M)$ for some unary primitive-recursive C_P, independent of H.*
2. *If $H(a, b) \le F(a)$ for all a, b, where F is a fixed unary primitive-recursive function, then P^H has a constant bound depending on P, F, and independent of H. The same statement holds with $F(b)$ in place of $F(a)$.*

Proof. For a fixed r-ary primitive-recursive operation g, put

$$\widehat{g}(z) = \max\{g(z_1, \ldots, z_r) : z_1, \ldots, z_r \le z\}.$$

This is a nondecreasing unary primitive-recursive function. Nullary operations are simply their constant values.

Traverse the finite acyclic circuit in a topological order. For the first assertion, give each numeral its constant bound, each oracle answer the bound M, and each ordinary operation node the bound $\widehat{g}$ of the maximum of its predecessor bounds. Every node bound is unary primitive recursive in M; the output bound is $C_P(M)$.

For the second assertion use constant bounds throughout the traversal. At an oracle query, apply the nondecreasing envelope $\widehat{F}(z) = \max_{t \le z} F(t)$ to the bound on the controlling query argument. Since its argument node precedes the query, its constant bound is already available. This gives a finite constant bound at every node. The proof for the second query coordinate is identical. □

Theorem 4.2 (Tied-seed circuit obstruction)**.** *Every fixed term $t(x_1, \ldots, x_k)$ of the language just described has either a constant bound or a bound*

$$t(x_1, \ldots, x_k) \le F(x_i)$$

for some one coordinate i and unary primitive-recursive function F. The coordinate and the function may depend on the term. This is an external standard-function statement; it makes no claim about a uniform primitive-recursive evaluator that, given arbitrary term syntax and an input value, computes the corresponding majorant value.

Proof. Replace every unary bounding function by its nondecreasing envelope. The property is preserved by a primitive operation with a known bound: only the term placed in its controlling input matters, and the two nondecreasing unary bounds can be composed. Constants and projections are immediate. It remains to check the new recursion constructor.

Apply the structural induction hypothesis to $h(u, v, \vec{p})$. Write $r_j(\vec{p})$ for its recursive state at counter j.

If $h(u, v, \vec{p}) \le F(p_i)$ for a passive parameter, the first circuit lemma gives

$$r_0(\vec{p}) \le C_P(F(p_i)), \qquad r_{j+1}(\vec{p}) \le F(p_i).$$

Consequently $\max(C_P(F(p_i)), F(p_i))$ bounds the result independently of the actual counter. A constant bound for h yields a constant bound in exactly the same way.

If $h(u, v, \vec{p}) \le F(u)$, the second circuit lemma bounds the initial state by a constant b. Every subsequent state satisfies $r_{j+1} \le F(j)$. Therefore the nondecreasing unary primitive-recursive function

$$M(n) = \max(b, F(n))$$

bounds r_n at every n, including $n = 0$.

If $h(u, v, \vec{p}) \le F(v)$, the second circuit lemma again gives a constant bound b on the initial state. Put

$$K(z) = \max(z, F(z)).$$

Then K is nondecreasing and $K \ge \mathrm{id}$. Induction on n gives $r_n \le K^n(b)$. Fixed-function iteration is primitive recursive, and $n \mapsto K^n(b)$ is nondecreasing.

In the last two cases the actual counter is itself a smaller term. Compose its constant or one-coordinate bound with M or $n \mapsto K^n(b)$, respectively. These cases exhaust the possible controlling coordinates of h, completing the structural induction. □

Corollary 4.3 (Combining two unbounded data inputs remains impossible)**.** *This term language cannot define ordinary binary addition, maximum, a general three-argument numerical selector, or an injection* $\mathbb{N}^2 \longrightarrow \mathbb{N}$.

Proof. For addition or maximum, hold the asserted controlling input fixed and increase the other input. For a selector $C(c, a, b)$ that returns a at $c = 0$ and b at $c > 0$, each proposed controlling coordinate is defeated by fixing it and increasing a selected data input. For an injection, fixing the controlling coordinate leaves infinitely many argument pairs whose outputs would all lie in one finite bounded interval. Constant bounds fail for the same reasons. □

What this does and does not establish. The original particular B operator is recovered by a one-query circuit followed by successor. The extension also rules out taking several fixed or adaptive probes of that same update, then adding, multiplying, maximizing, or otherwise primitively-recursively combining their answers to manufacture a seed. It continues to hold after adjoining arbitrary fixed unary primitive-recursive functions and the other bounded-input primitives described above. It is therefore stronger than checking a single seed recipe or searching a finite collection of candidate terms.

This remains a standard-function *term-definability* obstruction. It does not rule out quantified graph definitions, a relational interpretation, a different interpretation domain, additional axioms, or bi-interpretability obtained by another route. An independent unbounded initializer is one way to escape the hypothesis, but the theorem does not assert that it is the only way. The theorem is not a smaller positive axiomatization and is not presented as a globally novel result.

Why the interface restriction is necessary. If the initializer circuit is allowed one further independent passive input y, take $h(u, v, x, y) = x$ and let the initializer be $y + h(0, 0, x, y)$. At counter zero the resulting term is exactly $x + y$, so the one-coordinate conclusion fails. Likewise, a circuit permitted two independently chosen update oracles can query the constant updates $h_1 = x$ and $h_2 = y$ and add their answers. These observations identify the boundary of the theorem; they do not claim that either relaxation alone supplies a complete arithmetic basis or a smaller proof checker.

4.1 A boundary involving unbounded searches

The finiteness of the probe circuit matters. For an oracle $H(u, v; x, y)$ equal to 1 exactly when $(u, v) = (x, y)$ and 0 otherwise, every answer is bounded by 1. Nevertheless an unbounded search through all pairs eventually discovers (x, y) and can return $x + y$. Such a search is not a fixed finite query circuit; it is not total uniformly for all oracles either. This example explains why a claim about arbitrary oracle procedures would be false. Direct passive inputs, a separate second update oracle, or a relationally defined selection mechanism also require a separate analysis.

5 Finite tuple domains and quotient equality

The one-coordinate majorant theorem has a consequence that is stronger than failure to express ordinary addition as a single numerical term. It also obstructs interpretations using finite tuples and a nontrivial quotient equality, provided the interpreted operation is given by a tuple of terms. The following statement is purely about functions on the standard natural numbers; it does not require definability of the domain or the equivalence relation.

The underlying domination property is the established notion of an *almost unary function* on a countable set; its closure under composition belongs to the existing clone-theoretic literature [7]. The present argument uses that property and, for the recursor application, the stronger availability of primitive-recursive majorants. We give the finite-quotient result below with a complete proof and make no literature-wide priority claim for it.

Theorem 5.1 (Finite quotient from coordinate bounds)**.** *Let $d \geq 1$, let $D \subseteq \mathbb{N}^d$, let E be an equivalence relation on D, and let $e \in D$. Suppose $f : D \times D \longrightarrow D$ has a two-sided identity modulo E:*

$$f(e, z)\ E\ z, \qquad f(z, e)\ E\ z \quad (z \in D).$$

For each output coordinate i, suppose there are an input coordinate $j_i \in \{1, \ldots, d\}$ and a nondecreasing unary function g_i such that one of the following holds uniformly on $D \times D$:

$$f(x, y)_i \leq g_i(x_{j_i}), \qquad \text{or} \qquad f(x, y)_i \leq g_i(y_{j_i}).$$

A constant bound is also allowed. Then D/E is finite.

More explicitly, put $L(z) = f(e, z)$ and $R(z) = f(z, e)$. A composition of at most

$$\ell_d = \lceil \log_2(d + 1) \rceil$$

maps chosen from L, R sends every element of D to an E-equivalent representative in one fixed finite box $\prod_{i=1}^{d} \{0, \ldots, C_i\}$. If the coordinate bounds are primitive recursive, the constants and the word can be obtained effectively from those bounds, their selected coordinates, and e.

Proof. Under L, a coordinate bounded through an x input has the constant bound $g_i(e_{j_i})$; under R it is bounded by $g_i(z_{j_i})$. For a coordinate bounded through a y input, these roles are reversed. Thus each coordinate's bound is made independent of the unbounded input by at least one of the letters L, R. Under the other letter it depends on at most one coordinate. A coordinate with a constant bound is removed by both letters.

For precision, regard $\{1, \ldots, d\}$ as a finite set of states tracking dependencies of bounds, not actual functional dependencies. The transition for a letter sends state i either to the selected input state j_i, or to a terminal symbol † when that letter gives a constant bound. The terminal symbol is absorbing. Initially all d states are live. For a current live set S, the number of its states removed by L, plus the number removed by R, is at least $|S|$. Choose a letter removing at least $\lceil |S|/2 \rceil$ states, and replace the live set by the images of the surviving states. Each survivor has only one image, so

$$|S_{\text{next}}| \leq \lfloor |S|/2 \rfloor.$$

After at most ℓ_d choices the live set is empty, since $2^{\ell_d} > d$.

The letters are selected from the outside of the composition inward. If they are $U_1, \dots, U_k$, the resulting map is

$$W = U_1 \circ \cdots \circ U_k;$$

in an evaluation, U_k is applied first. This order matters for the dependency calculation. Inductively, each coordinate of the partial composition is either uniformly bounded, or bounded by a composition of the unary functions along one live dependency path. Appending the next letter on the inside replaces that live input coordinate by its new coordinate bound. Nondecreasingness permits this substitution of bounds. When the path reaches †, the resulting bound is a constant. The empty final live set therefore gives the constants $C_1, \dots, C_d$ asserted in the theorem.

Both L and R map D into D. The two identity laws say that each sends every argument to an E-equivalent element. Consequently, transitivity of E gives $W(z)\ E\ z$ for every $z \in D$. No compatibility axiom for f and E is needed for this step. Every E-class thus meets the finite set

$$D \cap \prod_{i=1}^{d} \{0, \dots, C_i\}.$$

In particular, the number of classes is at most $\prod_{i=1}^{d}(C_i + 1)$. □

An arbitrary unary bound may be replaced by its nondecreasing envelope $g^*(n) = \max_{m \le n} g(m)$. If g is primitive recursive, so is this envelope. The monotonicity condition therefore costs no additional assumption in the application to term majorants. For $d = 0$, the underlying tuple domain has at most one element and the conclusion is immediate.

Corollary 5.2 (No finite-dimensional term interpretation of iPRA)**.** *Suppose a numerical language has a standard model on $\mathbb{N}$ in which every term is bounded by a unary function of one of its free coordinates, or by a constant. No interpretation of iPRA in this model can use a fixed finite-dimensional domain, arbitrary quotient equality, and a term tuple for the interpreted addition operation. This remains true with fixed numerical parameters in the interpretation.*

Proof. Let $D \subseteq \mathbb{N}^d$ and E be the proposed interpretation domain and equality. Choose a representative $e \in D$ of its zero. The interpreted addition tuple satisfies the hypotheses of Theorem 5.1, because both zero identity laws are theorems of iPRA. A majorant in a fixed interpretation parameter is a constant bound and causes no exception. Hence D/E is finite.

On the other hand, iPRA's successor separation and injectivity give infinitely many distinct standard numerals in every model. Their interpretations would be infinitely many distinct E-classes. This contradicts the preceding finiteness conclusion. □

Only the two identity laws for addition were used in the finiteness argument; associativity, commutativity, and cancellation are unnecessary. Thus the obstruction applies more generally to an interpreted infinite unital magma. It does not rule out interpretations whose arithmetic operations are defined by quantified graphs instead of term tuples. Such relational interpretations are permitted by the original research question and remain outside this theorem.

5.1 Sharpness of the composition-length bound

The logarithmic word-length bound is attained in every positive dimension. This is a statement about compositions of the two unit maps, not about optimal proof-checker size or the shortest

possible definition of a representative map.

Fix $d \geq 1$, put $k = \lfloor \log_2 d \rfloor$, and first suppose $k \geq 1$. Of the d coordinates, label 2^k by the words $w \in \{L, R\}^k$. Write $\sigma(w)$ for the word obtained by removing its first letter and appending R. Take $D = \mathbb{N}^d$, let E be the universal equivalence relation, and let e be the zero tuple. For the labeled coordinates define

$$f(x, y)_w = \begin{cases} y_{\sigma(w)}, & w \text{ starts with } L, \\ x_{\sigma(w)}, & w \text{ starts with } R. \end{cases}$$

Set every remaining coordinate to zero. The unit laws hold modulo E, and every coordinate is a projection or a constant.

Under a unit-map letter, state w survives exactly when that letter is its first letter; in that case it moves to $\sigma(w)$. For any word $\alpha \in \{L, R\}^k$, the state initially labeled α therefore survives all k letters. In the actual composition its output coordinate is a projection of the input coordinate labeled R^k, so the composition is unbounded. The letters here are again read from the outside of the composition inward. Hence no word of length k gives a uniformly bounded map; a shorter bounded word would remain bounded when extended on the inside and is equally impossible.

The word L^{k+1} removes every dependency. A state containing R in one of its original k positions is removed on reaching that position. The only state surviving the first k letters is L^k, which has then become R^k and is removed by the last L. Constant coordinates are removed immediately. Thus the minimum length is exactly

$$k + 1 = \lceil \log_2(d + 1) \rceil .$$

If $k = 0$, then $d = 1$. Set $f(x, y) = x$, use universal E, and take $e = 0$. The map R is the identity and L is constant zero. The empty word is unbounded and a single L is bounded, giving the required exact minimum one in this case too.

5.2 The limits of the obstruction are concrete

Both identity laws are necessary. On $D = \mathbb{N}$ with actual equality, the projection $f(x, y) = x$ has $f(z, e) = z$ for every fixed e and satisfies the one-coordinate bound, but D/E is infinite. The omitted identity law is exactly what fails. The opposite projection illustrates the other one-sided case.

Relational definitions also give a direct escape from the term hypothesis. In a first-order setting with decidable equality to zero, the quantifier-free graph

$$M(x, y, z) \;\equiv\; (x = 0 \wedge z = y) \;\vee\; (x \neq 0 \wedge z = x)$$

is total and single-valued. It defines the operation that returns y when $x = 0$ and otherwise returns x. Its two-sided identity is zero, so it is an infinite unital magma on $\mathbb{N}$. Indeed it is associative: it selects the first nonzero argument in a finite list, or zero if all are zero. It cannot be represented by an almost unary numerical term, but its graph is already a finite logical formula.

This example is not an interpretation of iPRA: no addition compiler or closure under primitive recursion follows from it. It demonstrates that the remaining relational route is a substantive possibility. The finite-quotient theorem does not close that route or prove that the requested smaller axiomatization cannot exist.

6 An exact obstruction to removing recursive major proofs

A different proposed simplification replaces recursive checking of every elimination major by membership in the retained context. Under the current public Eq/Q/D interface, that deletion loses a theorem before arithmetic or induction enters the argument.

Proposition 6.1. *The inherited calculus proves $Q_y(0 = 0)$. A calculus with only retained- hypothesis elimination majors, computation supplies adding only equation hypotheses, the same paired introductions, and optional direct induction at equation goals does not prove this closed formula, provided no separate unrestricted cut rule is added.*

Proof. Under the assumption $0 = 0$, Hyp proves $0 = 0$. DI-case with witness zero therefore proves $D_{x,y}(0 = 0)$. DE applied to that derived major supplies $Q_y(0 = 0)$ as a local hypothesis; Hyp then closes the required goal.

For the restricted calculus, start from a Q-root goal with an equation- only context, initially empty. Hyp cannot close it. A paired introduction concludes D, and computation supply or direct equation induction cannot introduce a Q hypothesis. Transport using an available equation preserves the Q root of its minor. QE and DE would require a Q or D retained hypothesis, neither of which is present. An induction on finite derivation height therefore excludes such a proof. □

For the unchanged flat-computation checker, the entire canonical JSON certificate is

```
[[[],0,0,0],[[0,0,[0,0]],0,0,0],[[0,0]]]
```

It is accepted both normally and with Python optimization. The deliberately restricted checker replaces precisely

```
        c(m, g)
```

by

```
        if i(m) not in g:
            raise ValueError("Elimination major is not a retained hypothesis")
```

and rejects the certificate. Seven theorem/control cases across both checkers and both Python modes give 28 successful expected-result runs. The exact mutation, source identities, stdout, stderr, and exit codes are preserved in `tests/normalization/`.

This is a theorem about the stated deletion and public interface. It is not a proof that Q must remain an object connective: the existing Eq/D presentations can make Q an internal closure resource, with Q_yB represented using a dummy witness in $D_{x,y}B$. A redesigned calculus must establish its own reconstruction theorem. Explicitly, $D_{u,y}B \to Q_yB$ (with u unused) is DE followed by Hyp, and the converse follows by DI-case at witness zero: from Q_yB and the local $B(0)$, QE proves $B(Sz)$. An unrestricted cut would also repair the displayed theorem by inserting the derived D formula into the context; the old cut-admissibility argument cannot simply be imported after restricting major premises.

Even after excluding public Q goals, ordinary logical normalization alone does not justify deleting all derived majors. Equation induction can produce a derived equality subsequently used for transport into a compound formula. At an arbitrary numerical parameter that induction is neutral, rather than

a reducible successor case. A proposed commuting conversion must not silently invoke induction on the compound transport motive. This identifies an additional proof obligation, not a blanket incompleteness theorem for every redesigned Eq/D system.

7 Validation, reproducibility, and limits

7.1 What was checked

The packaged reproduction recomputes the hashes and whole-source metrics of all 297 distinct Python files without executing them. It checks every recorded source occurrence against the matching hash and recomputes the coordinate envelope. The selected gzip streams are reproduced at compression level 9 with timestamp zero.

The existing byte-leading checker was independently replayed on its 28 supplied witness cases and three malformed inputs in ordinary and optimized Python: 62 runs, all with the expected public verdict and exit status. The Q-theorem deletion experiment adds 28 process runs. The finite-dependency algorithm in `code/unit_collapse.py` exhaustively checks small systems and the explicit sharp family; its exact counts are recorded in `evidence/unit_collapse_results.json`.

The mathematical obstructions were checked independently, in particular the monotone-envelope requirement, outer-to-inner composition order, possibility that dependency paths merge, quotient equivalence without congruence, and the limitation to the standard source model. These are written arguments supported by diagnostic code, not proof-assistant certifications or external peer review.

7.2 What the results do not establish

No new complete checker is below the three current normalized coordinate minima. The failed local-scope prototype has 685 bytes, 279 tokens, and 322 AST nodes against its matched predecessor's 684/277/317. Its raw gzip size drops from 403 to 400, but that informational improvement does not justify selecting it. It is retained as a rejected experiment, not as a complete verified replacement.

The new arithmetic obstruction applies to the precisely stated update- query family. The finite-quotient theorem excludes operations represented by tuples of its numerical terms. General first-order interpretations may instead define operation graphs using formulas and witnesses; those are still available possibilities. Accordingly, there is no conclusion here that all smaller axiomatizations are impossible or that all bi-interpretations using a different domain have been excluded.

8 Further research questions

1. **Can a relational presentation exploit the remaining escape?** The quotient theorem eliminates a broad collection of term encodings. A concrete next target is an interpretation whose operations are given by uniquely determined graphs, together with a proof of their totality and both interpretation composites. Defining a selector by cases does not automatically permit recursion over update bodies containing it; that extension needs its own totality proof.
2. **What is the cheapest way to leave the almost unary class?** An independent seed is sufficient in the existing system. The question is whether another operation that combines two unbounded inputs can replace it while reducing the complete checker, including its defining

equations and scope rules. The no-pairing and finite-quotient theorems give tests that a proposed replacement must pass.

3. **Can atomic cuts be compiled away without compound induction?** A positive normalization theorem for derived equality majors might permit a smaller focused checker. It must handle equation-induction proofs used for arbitrary formula transport. An ordinary natural- deduction normalization slogan is insufficient.
4. **Can the native codec and its interpretation be formalized once?** The protected-slot opener, fresh-parameter conditions, malformed-data contract, and canonical certificate serialization are reusable formalization targets. Their mechanization would distinguish source minimization from changes in what the checker actually accepts.
5. **Does the quotient obstruction extend to graph interpretations?** It cannot extend without additional hypotheses: a relation can define an operation with no almost-unary term representative. A useful theorem would identify an exact witness-extraction property under which graph interpretations reduce to the term case, or provide a small construction showing how the remaining freedom can be used for iPRA.

A Source map and exact benchmark programs

The archive contains complete sources rather than relying on changing branch URLs. The following names are package-local aliases of unchanged pinned sources; original paths and SHA-256 values are in `evidence/selected_sources.json` and `evidence/selected_metrics.json`.

Local reference	Original role
`byte_leader.py`	Counter-congruence ancillary zero-product
`token_leader.py`	Constructed-computation explicit checker
`ast_leader.py`	Transient-seed structure profile
`raw_leader.py`	Local-rewriting scalar boxed checker
`gzip_leader.py`	Local-rewriting balanced checker
`joint_profile.py`	Transient-seed joint profile
`named_reference.py`	Constructed-computation readable implementation

For direct inspection, here is the entire existing 672-byte program. Its brevity is not attributed to the new obstruction theorem.

```
import json
k = 0
i = lambda a, t=None, d=0: [i(b, t, d + b' \x02\x01\x04'[len(a)]) for b in a] if a * 0 == [] else
     a if k <= a < d else d % 2 and i(t, d=1)

def c(a, g):
    global k
    if i(a) in g:
        return
    m, s, u, t = q()
    if m:
        c(m, g)
        if m in ([s, u], not s and [t]) and i(t, u) == a:
            a = i(t, s)
        h = m[2:]
    elif t:
```

```
        h = a = i(a[2], t, 2)
    else:
        n, t, u = s
        h = [s, i(u, i([*n, t, u], d=1), 1) if n else t]
    k -= 1
    c(i(a, [k]), g + [i(h, k if m or len(a) % 2 == m else 0)])
try:
    q = json.load(open(0)).pop
    c(q(), [])
except:
    print('REJECT')
    raise
print('ACCEPT')
```

B Reproduction commands

Run from the extracted package directory, using the recorded Python environment when byte-identical results are required:

```
python3 reproduce.py
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

The Python scripts use only the standard library. A source-normalizer or compression-runtime difference is a reason to report a new measurement, not silently relabel it as an exact reproduction. The corpus files are included for inspection and parsing; they are not an installation bundle and need not be executed.

References

[1] VladimirReshetnikov/ProveIt, *docs/drafts*, commit `0300a290bedee5019c974291850b798378438b2e`. https://github.com/VladimirReshetnikov/ProveIt/tree/0300a290bedee5019c974291850b798378438b2e/docs/drafts.

[2] ProveIt draft, *Counter Congruence Suffices for iPRA*, `ipra_counter_congruence_2026-11-10.zip`, in [1]. Specific tied-seed majorant theorem and ancillary comparison programs.

[3] ProveIt draft, `ipra_transient_seed_2026-11-10.zip`, in [1], `theory/obstruction.tex`. Specific tied-seed theorem, composition closure, and no-pairing corollary.

[4] ProveIt draft, *Constructing Computation Sources*, `ipra_constructed_computation_2026-11-10.zip`, in [1]. Complete inherited presentation, interpretations, and explicit checker.

[5] ProveIt draft, `ipra_flat_computation_2026-11-10.zip`, in [1]. Unchanged public Eq/Q/D checker used for the exact deletion experiment.

[6] S. Salehi, *Decidable formulas of intuitionistic primitive recursive arithmetic*, Reports on Mathematical Logic 36 (2002), 55–61, Section 1. https://rml.tcs.uj.edu.pl/rml-36/36-SALEHI.pdf.

[7] M. Pinsker, *Clones containing all almost unary functions*, arXiv:math/0401102 (2004), Section 1. https://arxiv.org/abs/math/0401102. Author's manuscript: https://www.dmg.tuwien.ac.at/pinsker/papers/interval/interval.pdf.